Neural Networks - Part 2

Machine Learning and Big Data - DATA622

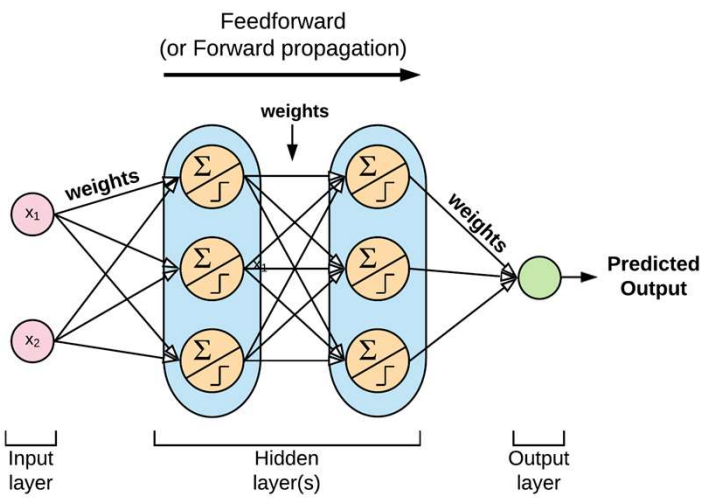
CUNY School of Professional Studies

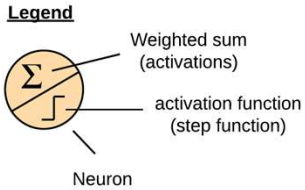
Feedforward

How neural networks make predictions

CUNY School of Professional Studies

Feedforward

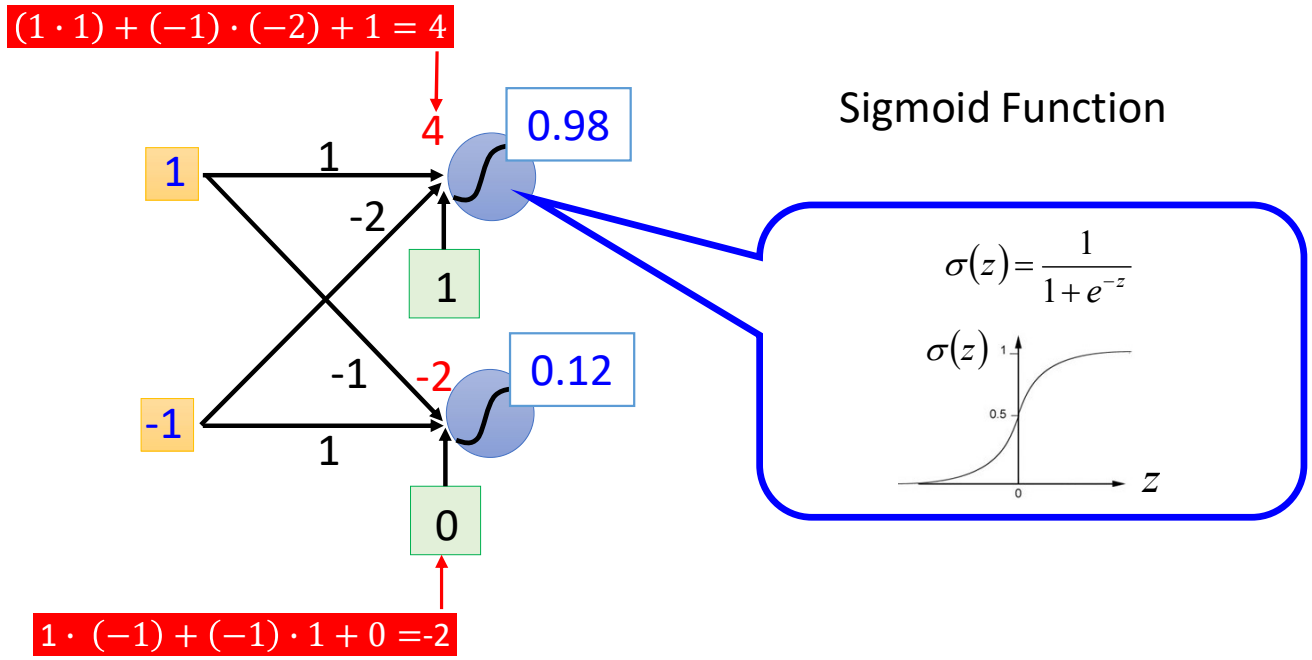




Source: https://ekababisong.org/gcp-ml-seminar/deep-learning/

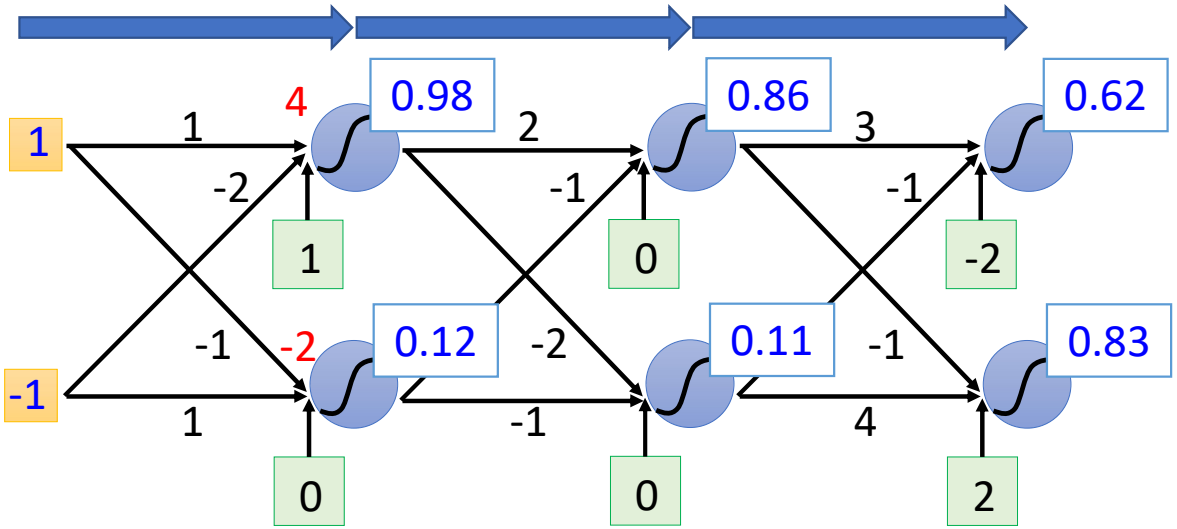
Feedforward

- A simple network, toy example



Source: Hung-yi Lee – Deep Learning Tutorial

Feedforward



$$f: R^2 \rightarrow R^2 \qquad f\left(\begin{bmatrix}1\\-1\end{bmatrix}\right) = \begin{bmatrix}0.62\\0.83\end{bmatrix}$$

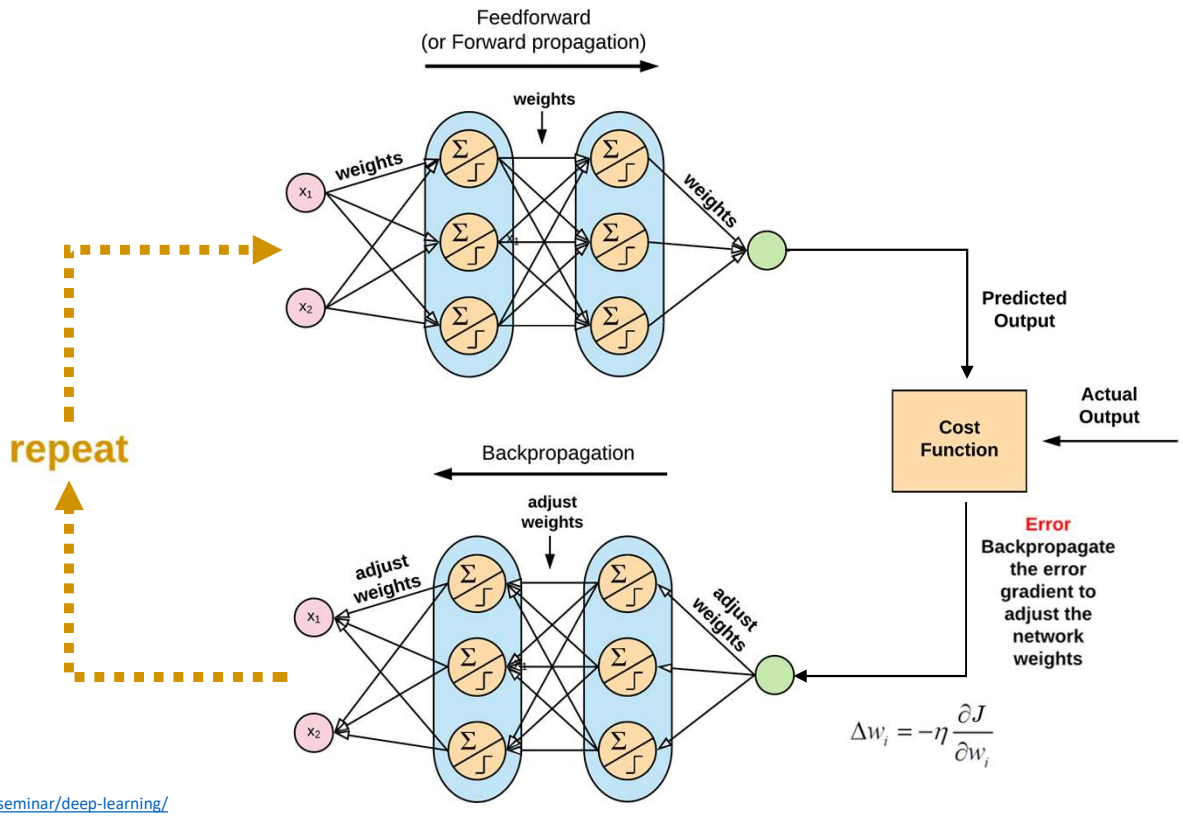
Source: Hung-yi Lee – Deep Learning Tutorial

Backpropagation

How neural networks are trained

CUNY School of Professional Studies

Backpropagation



Source: https://ekababisong.org/gcp-ml-seminar/deep-learning/

Backpropagation

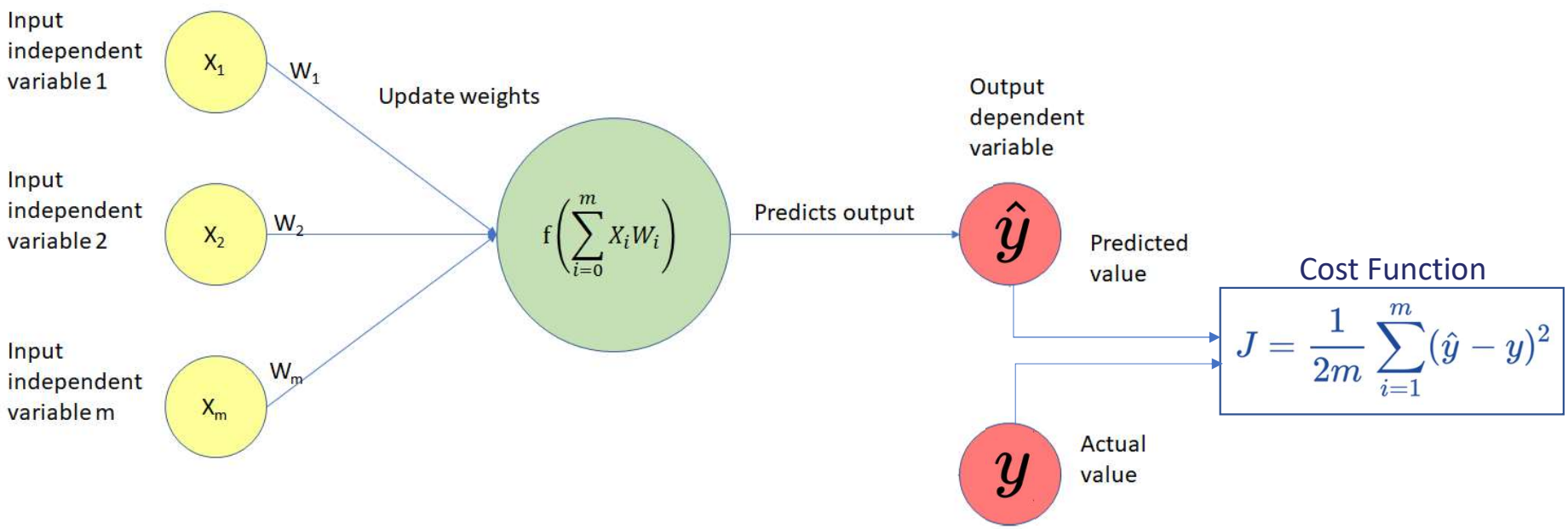
- Backpropagation is a type of gradient descent (terms used interchangeably)
- Gradient descent refers to the calculation of a gradient on each weight in the neural network for each training element.
  - Because the neural network will not output the expected value for a training element, the gradient of each weight will give you an indication about how to modify each weight to achieve the expected output.
  - If the neural network did output exactly what was expected, the gradient for each weight would be 0, indicating that no change to the weight is necessary.
- How it works:
  - Output of NN is evaluated against desired output
  - If results are not satisfactory, connection (weights) between layers are modified and process is repeated again and again until error is small enough.

Cost function

Tracking how your training is performing

CUNY School of Professional Studies

Cost Function (J)



Cost Function

- Cost functions determine how well a model performs for a given dataset
- Cost Functions measure just how wrong the model is in finding a relation between the input and output.
- Comparing other functions:
  - Accuracy functions tell you how well your model does, not how to improve it
  - Error functions measure the difference between the target and the actual values e.g. $(\hat{y} - y)$
  - Loss functions quantify the cost for a single training example e.g. $(\hat{y} - y)^2$
  - Cost functions quantify the average across the entire dataset
    e.g.
    $$J = \frac{1}{2m} \sum_{i=1}^{m} (\hat{y} - y)^2$$
  - Loss functions quantify the impact of the error i.e. error is objective while loss is subjective e.g. we might adopt a non-symmetric loss function if we may be more negatively affected by an error in a particular direction (e.g., false positive vs. false negative)

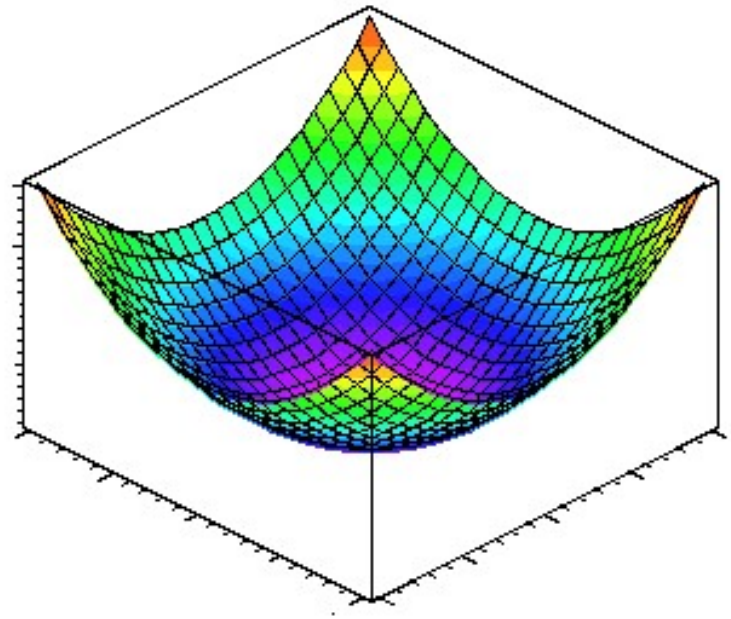
Cost Function

- MSE Cost function will always be parabolic (by definition)
  - So it has only one global minimum

$$J = \frac{1}{2m}\sum_{i=1}^{m}(\hat{y} - y)^2$$

- SSE – Similar to MSE but not an average

$$J = \frac{1}{2}\sum_{i=1}^{m}(\hat{y} - y)^2$$



Note: The 2 we added in the denominator makes the mathematics cleaner.

Gradient Descent Rule

Gradient descent rule:

$$w_i \leftarrow w_i + \Delta w_i$$

Where:

$$\Delta w_i = -\eta \frac{\partial J}{\partial w_i}$$

η is a positive constant called the *learning rate*, and determines step size of gradient descent search

CUNY School of Professional Studies

Cost Function

$$SSE = \frac{1}{2}\sum_{j=1}^{n}\left(y_j - \hat{y}_j\right)^2$$

$$SSE = \frac{1}{2}\sum_{j=1}^{n}\left(y_j - \left(\sum_{i=0}^{m}\phi\left(w_i^T \times x_{j,i}\right)\right)\right)^2$$

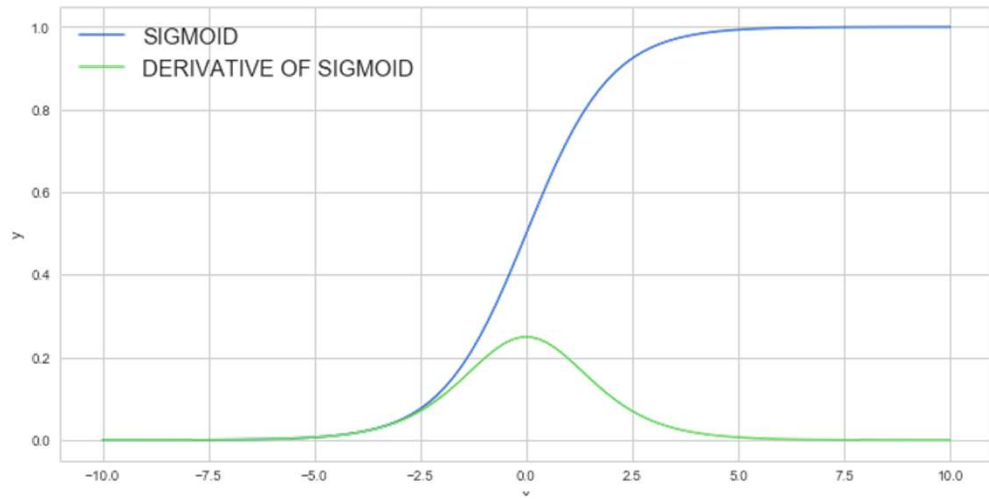
$$\frac{\partial\, SSE}{\partial\, w_i} = \sum_{j=1}^{n}\left(\underbrace{\left(y_j - \hat{y}_j\right)}_{\substack{\textit{error of the}\\ \textit{output of the}\\ \textit{weighted sum}}} \times \underbrace{-x_{j,i}}_{\substack{\textit{rate of change of}\\ \textit{weighted sum}\\ \textit{with respect to}\\ \textit{change in } w_i}}\right)$$

Derivative of the Sigmoid Function

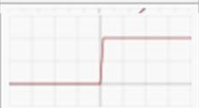
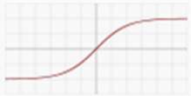
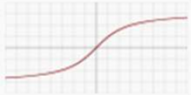
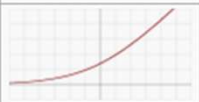
$$y = \frac{1}{1+e^{-x}}$$

$$\frac{dy}{dx} = -\frac{1}{(1+e^{-x})^2}(-e^{-x}) = \frac{e^{-x}}{(1+e^{-x})^2}$$

$$= \frac{1}{1+e^{-x}}\left(1 - \frac{1}{1+e^{-x}}\right) = y(1-y)$$



Derivatives of other Activation Functions

Name	Plot	Equation	Derivative
Binary step		$f(x) = \begin{cases} 0 & \text{for} \quad x < 0 \\ 1 & \text{for} \quad x \geq 0 \end{cases}$	$f'(x) = \begin{cases} 0 & \text{for} \quad x \neq 0 \\ ? & \text{for} \quad x = 0 \end{cases}$
TanH		$f(x) = \tanh(x) = \frac{2}{1+e^{-2x}} - 1$	$f'(x) = 1 - f(x)^2$
ArcTan		$f(x) = \tan^{-1}(x)$	$f'(x) = \frac{1}{x^2+1}$
Rectified Linear Unit (ReLU)		$f(x) = \begin{cases} 0 & \text{for} \quad x < 0 \\ x & \text{for} \quad x \geq 0 \end{cases}$	$f'(x) = \begin{cases} 0 & \text{for} \quad x < 0 \\ 1 & \text{for} \quad x \geq 0 \end{cases}$
Parameteric Rectified Linear Unit (PReLU) [2]		$f(x) = \begin{cases} \alpha x & \text{for} \quad x < 0 \\ x & \text{for} \quad x \geq 0 \end{cases}$	$f'(x) = \begin{cases} \alpha & \text{for} \quad x < 0 \\ 1 & \text{for} \quad x \geq 0 \end{cases}$
Exponential Linear Unit (ELU) [3]		$f(x) = \begin{cases} \alpha(e^x - 1) & \text{for} \quad x < 0 \\ x & \text{for} \quad x \geq 0 \end{cases}$	$f'(x) = \begin{cases} f(x) + \alpha & \text{for} \quad x < 0 \\ 1 & \text{for} \quad x \geq 0 \end{cases}$
SoftPlus		$f(x) = \log_e(1 + e^x)$	$f'(x) = \frac{1}{1+e^{-x}}$

Cost Function

$$SSE = \frac{1}{2}\sum_{j=1}^{n}\left(y_j - \hat{y}_j\right)^2$$

$$SSE = \frac{1}{2}\sum_{j=1}^{n}\left(y_j - \left(\sum_{i=0}^{m}\phi\left(w_i^T \times x_{j,i}\right)\right)\right)^2$$

$$\frac{\partial\, SSE}{\partial\, w_i} = \sum_{j=1}^{n}\left(\underbrace{\left(y_j - \hat{y}_j\right)}_{\substack{\textit{error of the}\\ \textit{output of the}\\ \textit{weighted sum}}} \times \underbrace{-x_{j,i}}_{\substack{\textit{rate of change of}\\ \textit{weighted sum}\\ \textit{with respect to}\\ \textit{change in } w_i}}\right)$$

Gradient Descent Rule or Sigmoid

$$w_i^{t+1} = w_i^t + \left(\underbrace{\eta \times \sum_{j=1}^{n} \left(\left(y_j^t - \hat{y}_j^t\right) \times \underbrace{\left(\hat{y}_j^t \times \left(1 - \hat{y}_j^t\right)\right)}_{\text{Derivative of the sigmoid activation function with respect to the weighted sum}} \times x_{j,i}^t \right)}_{\text{Error gradient for } w_i} \right)$$

Cost Function

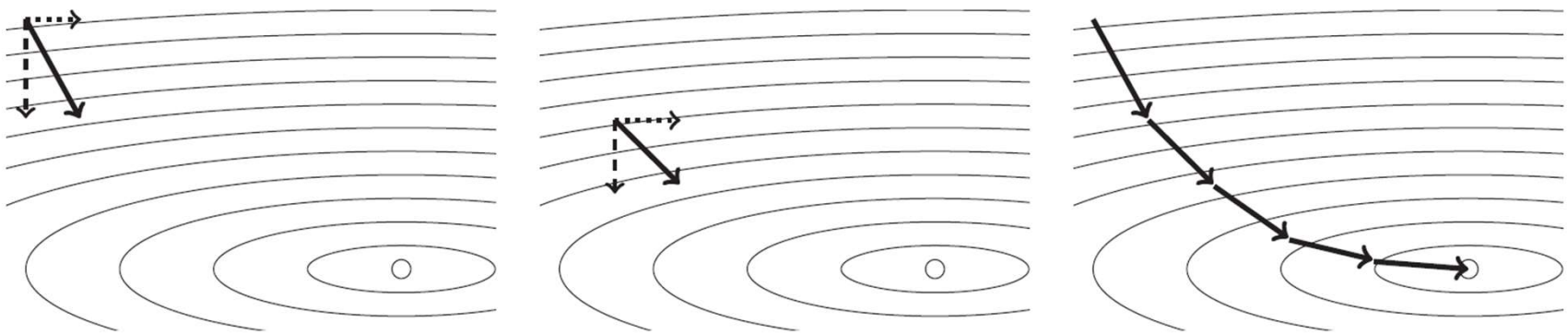
$$w_i^{t+1} = w_i^t + \left(\eta \times \underbrace{\sum_{j=1}^{n} \left(\left(y_j^t - \hat{y}_j^t \right) \times x_{j,i}^t \right)}_{\text{error gradient for } w_i} \right)$$

Learning Rate

You need to adjust the learning rate as you train

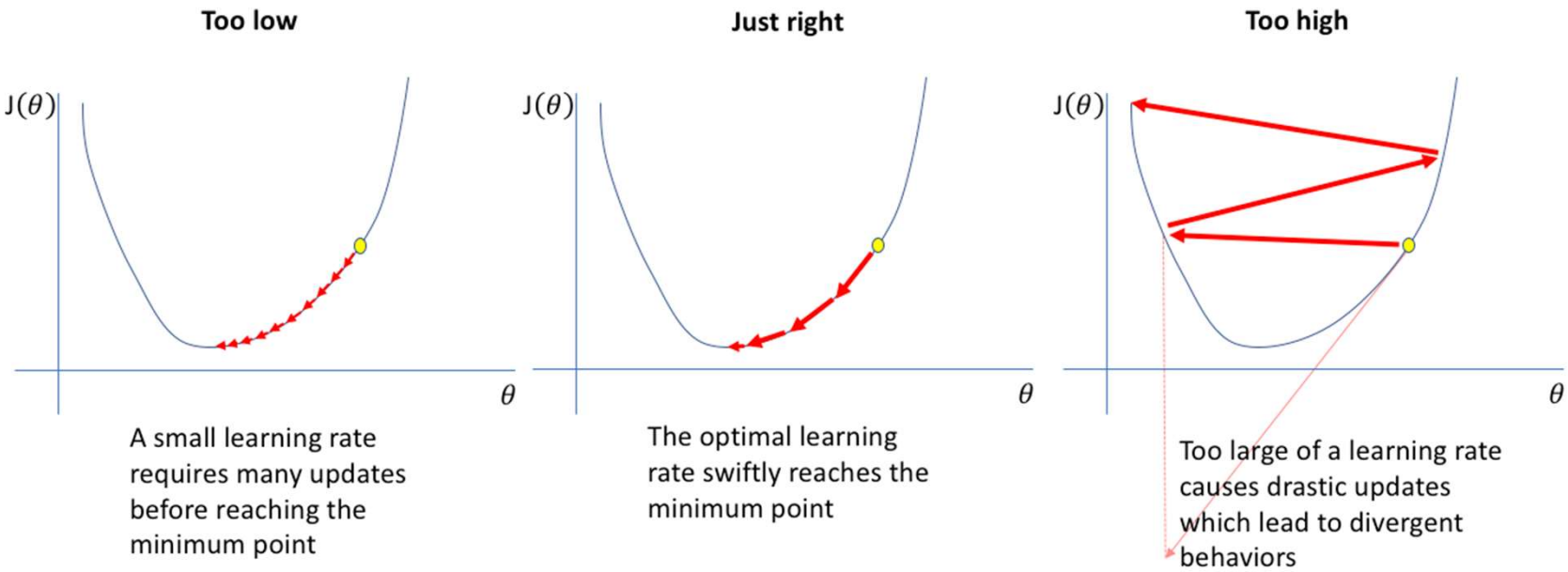
Gradient descent

The gradient tells us the direction in which the loss has the steepest rate of increase, but it does not tell us how far we should step



Learning Rate

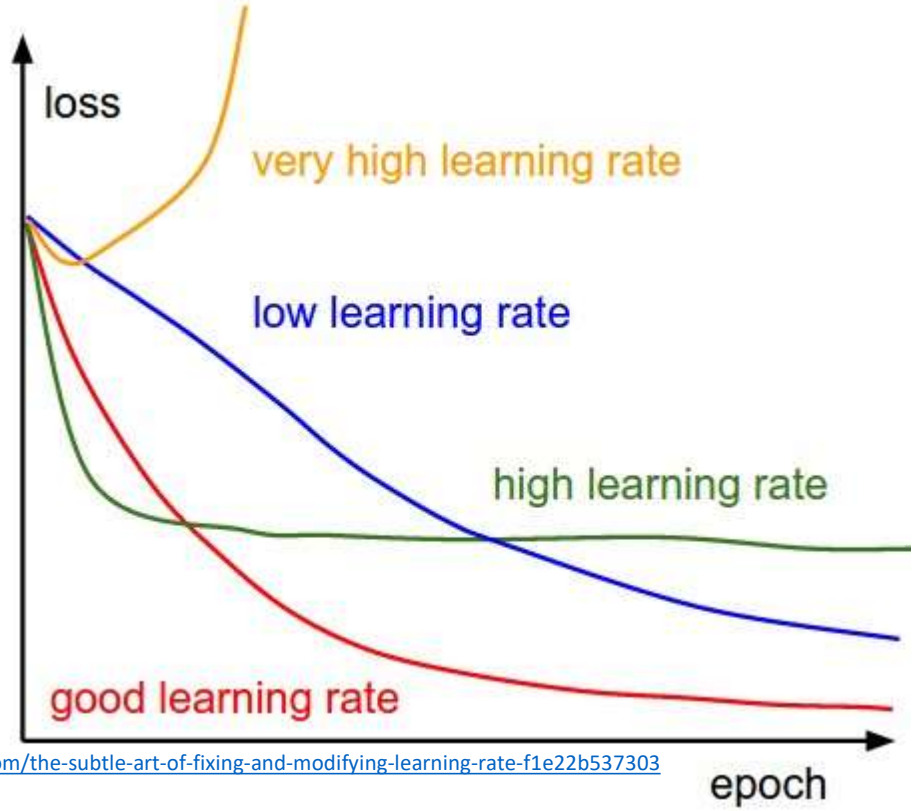
Learning rate (step size) is a hyper-parameter which defines the speed of 'learning'



Source: https://www.jeremyjordan.me/nn-learning-rate/

Impact of different learning rates

- High learning rate: loss increases or plateaus too quickly
- Low learning rate: loss decreases too slowly: too many epochs to reach a solution

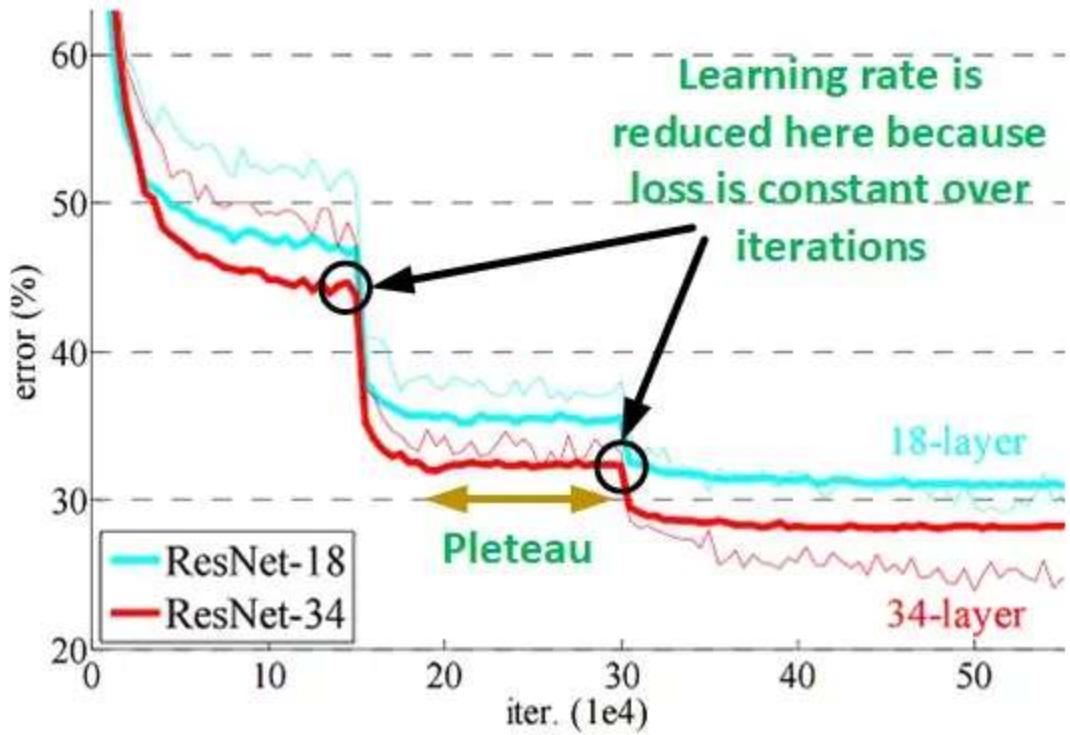


Source: https://towardsdatascience.com/the-subtle-art-of-fixing-and-modifying-learning-rate-f1e22b537303

Adjusting learning rates

- Learning rates must be decayed during training
- Approach 1: Automatic Decay
  - Reduce the learning rate by some factor every few epochs
  - Typical values: reduce the learning rate by a half every 5 epochs, or by 10 every 20 epochs
  - Exponential decay reduces the learning rate exponentially over time
  - These numbers depend heavily on the type of problem and the model
- Approach 2: Plateau-driven
  - Reduce the learning rate by a constant (e.g., by half) when the validation loss stops improving
  - In TensorFlow: tf.keras.callbacks.ReduceLROnPleateau()
  - Monitor: validation loss
  - Factor: 0.1 (i.e., divide by 10)
  - Patience: 10 (how many epochs to wait before applying it)
  - Minimum learning rate: 1e-6 (when to stop)

Adjusting learning rates



Source: https://towardsdatascience.com/the-subtle-art-of-fixing-and-modifying-learning-rate-f1e22b537303